第5章 自主机器人的规划与控制系统

智能技术学部

2026年4月

目录

01 规划与控制概述

02 路径规划和避障

2.1 路径规划
2.2 轨迹规划
2.3 变分方法
2.4 图搜索方法
2.5 增量搜索策略

概述

本次课程以基本硬件承载全技术链，解决算法-硬件-场景的具体工程问题，培养同学们在具身智能领域的动手能力，涉及机器人自主运动的核心技术链条，聚焦自主机器人开发范式，以ROS机器人为硬件平台，贯通 "感知-规划-控制"全技术链，目标是在动态环境中实现机器人的自主运动控制能力，为同学们搭建从理论到具身产业的桥梁。

路径规划和避障

路径规划与轨迹规划

实践目标1：路径规划/轨迹规划二选一，路径规划任务是在栅格地图中实现A*全局路径规划；轨迹规划任务是将路径点转为时间-速度轨迹，并优化平滑性，在实践过程中随时进行知识补充。

路径规划和避障

路径规划目标：在栅格地图中实现A*全局路径规划

开始动手！

操作流程：

1. 建图与地图加载：

```
# 启动激光SLAM建图命令
ros2 launch slam_toolbox online_async_launch.py

2. A*算法实现（Python）：
# 关键代码：启发式搜索
def astar(grid, start, goal):
    open_set = PriorityQueue()
    open_set.put((0, start))
    came_from = {}  # 路径回溯字典
    g_score = {cell: float('inf') for cell in grid}
    g_score[start] = 0
    while not open_set.empty():
        current = open_set.get()[1]
        if current == goal:
            return reconstruct_path(came_from, goal)
for neighbor in get_neighbors(current):
            tentative_g = g_score[current] +
distance(current, neighbor)
            if tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f_score = tentative_g +
heuristic(neighbor, goal)
                open_set.put((f_score, neighbor))
```

A*算法：
https://blog.csdn.net/dujuancao11/article/details/109749219

路径规划和避障

轨迹规划与变分优化目标：将路径点转为时间-速度轨迹，并优化平滑性

开始动手！

操作流程：

1.基础轨迹生成（梯形速度曲线）：

```
def trapezoidal_profile(path, max_vel, max_accel):
    # 计算每个路径段的速度轮廓
    for i in range(len(path)-1):
        dist = distance(path[i], path[i+1])
        t_accel = max_vel / max_accel #加速段时间
        s_accel = 0.5 * max_accel * t_accel**2
        if dist < 2*s_accel: #距离不足时调整最大速度
            actual_max_vel = math.sqrt(max_accel *
dist)
            # ... 分段计算时间戳
        else:
            # 完整加速-匀速-减速三段
    return time_stamped_trajectory
```

2.变分方法优化（梯度下降平滑）：

```
def smooth_trajectory(initial_path,
obs_cost_weight, smooth_weight):
    # 迭代调整点位置以降低代价
    for _ in range(iterations):
        for i in range(1, len(initial_path)-1):
            # 计算梯度：平滑项 + 碰撞惩罚项
            grad =
smooth_weight*(initial_path[i] -
(prev+next)/2)
                    +
obs_cost_weight*obstacle_gradient(initial_
path[i])
            initial_path[i] -= learning_rate *
grad
    return initial_path
```

变分方法：
https://zhuanlan.zhihu.com/p/341408183

路径规划和避障

图搜索方法与增量搜索策略

实践目标2：图搜索方法/增量搜索策略二选一，实践Dijkstra算法和Anytime RRT*算法，在实践过程中进行知识的补充。

路径规划和避障

图搜索方法与增量搜索策略

开始动手！

```
#Dijkstra算法实现（栅格地图）：
# 关键步骤：代价均匀扩散
open_set = PriorityQueue()
open_set.put((0, start))  # (代价, 节点)
while open_set:
    current_cost, current = open_set.get()
    for neighbor in neighbors:
        new_cost = current_cost + uniform_cost  # 所有边代价相同
        if new_cost < cost_so_far[neighbor]:
            cost_so_far[neighbor] = new_cost
            open_set.put((new_cost, neighbor))
```

路径规划和避障

图搜索方法与增量搜索策略

开始动手！

```
# 核心操作：随机采样+父节点重连接
random_point = sample_free_space()
nearest = find_nearest(tree, random_point)
new_point = steer(nearest, random_point, step_size)
if collision_free(nearest, new_point):
    near_nodes = find_near(tree, new_point, radius)
    choose_parent(new_point, near_nodes)  # 选择最优父节点
    rewire(tree, new_points, near_nodes)  # 重连优化路径
```

探索实践：如何实现受限算力条件下（如嵌入式开发板可用资源低）搜索算法的复杂度优化？（独立实践考察）

谢 谢！

智能技术学部

2026年